
Speculative Multi-Token Generation with Linguistically-Informed Constraints for Indian Languages

CS798: MTP Stage II Project Report

*Submitted in partial fulfillment of requirements for the degree
of*

Masters of Technology

By

Pranav Shinde

Roll No.: 23M0833

under the guidance of

Prof. Ganesh Ramakrishnan



Department of Computer Science and Engineering
INDIAN INSTITUTE OF TECHNOLOGY, BOMBAY

JUNE, 2026

Approval Sheet

This dissertation entitled ”**Speculative Multi-Token Generation with Linguistically-Informed Constraints for Indian Languages**” by **Pranav Jagadip Shinde** is approved for the degree of Masters of Technology in Computer Science and Engineering.

Supervisor

Examiners

Chairman

Place:

Date:

Declaration

I declare that this written submission represents my ideas in my own words, and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated, or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will cause for disciplinary action by the Institute and can evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Date: 23/06/2026

Place: IIT Bombay, Powai, Mumbai

Pranav Shinde

Roll No: 23M0833

Acknowledgements

I am thankful to my supervisor **Prof. Ganesh Ramakrishnan** for his continuous support, conviction, encouragement, and invaluable advice in this thesis work. I also would like to thank **Karthika N. J., Gokul Gopinathan** and **Hrishikesh Terdalkar** for helping me through the background and updates.

Abstract

Large Language Models (LLMs) suffer from inference latency due to their token-by-token generation process. Speculative decoding mitigates this by drafting and verifying multiple tokens in parallel, yet existing methods often ignore the morphological and syntactic richness of languages like those of India. Such oversight leads to fragmented tokenization, inefficiency, and loss of coherence.

This thesis proposes a linguistically informed speculative decoding approach that integrates constraints from the Samanvaya framework to enhance both efficiency and semantic integrity in Indian languages. We introduce a Linguistically-Constrained Suffix Automaton (SAM) that employs Local Word Groups (LWGs)—minimal semantic units—to guide retrieval-based speculative drafting. By ensuring drafted spans remain within a single LWG, our method prevents incoherent continuations and prunes the search space effectively.

We extend a state-of-the-art retrieval-based speculative decoding framework (SAM-Decoding), embedding LWG boundaries into its static suffix automaton. During inference, the drafter generates linguistically valid spans, which are then verified for acceptance.

This work advances semantically grounded and efficient LLM inference for low-resource, structurally diverse languages.

Contents

Approval Sheet	i
Declaration	ii
Acknowledgements	iii
Abstract	v
1 Introduction	1
1.1 The Rise of Large Language Models and the Inference Challenge	1
1.2 The Unique Landscape of Indian Languages	2
1.3 Problem Statement	2
1.4 Proposed Solution: A Linguistically-Informed Approach . . .	3
1.5 Key Contributions	3
2 Background and Literature Review	5
2.1 Speculative Decoding: The Drafter-Verifier Paradigm	5
2.2 State-of-the-Art in Speculative Decoding	6
2.3 The Suffix Automaton: An Engine for Pattern Retrieval . . .	9
2.4 The Principle of "Semantically Cohesive Word Grouping" . . .	12
3 Data Preparation: Creating a Linguistically-Informed Corpus	15
3.1 The Need for Boundary-Aware Data	15
3.2 Leveraging the Samanvaya Framework for Annotation	16
3.2.1 The Rule-Based Word Grouping Process	17

3.2.2	Handling Inflectional, Derivational, and Compound Word Groups	17
3.3	Designing the Data Structure	20
3.3.1	Representing Tokens and Boundaries	21
3.3.2	Example of an Annotated Sentence	21
3.4	Corpus Curation and Preprocessing Pipeline	22
4	Methodology: The Word-Group-Aware SAM (WGA-SAM) Framework	24
4.1	Architectural Overview	24
4.2	Core Innovation: Modifying the Suffix Automaton	27
4.2.1	Extending the Data Structure with Boundary Information	27
4.2.2	The Boundary-Aware build Algorithm	27
4.2.3	Aggregating Multiple End-Positions for Draft Diversity	28
4.3	Advanced Draft Generation: From Single to Multiple Candidates	29
4.3.1	The Modified <i>gen_draft</i> Algorithm	30
4.3.2	Generating Multiple Draft Candidates via Top-k End-Positions	30
4.4	The Complete Verifier-Drafter Workflow	31
4.4.1	The Foundational Single-Draft Workflow	31
4.4.2	The Advanced Multi-Draft Workflow with Candidate Verification	32
5	Experimental Setup and Results	34
5.1	Experimental Setup	34
5.2	Results and Analysis	36
6	Conclusion and Future Directions	40
6.1	Summary of Research	40
6.2	Limitations of the Current Work	41
6.3	Future Research Directions	42

Chapter 1

Introduction

1.1 The Rise of Large Language Models and the Inference Challenge

The advent of Large Language Models (LLMs) such as the GPT series [Brown et al., 2020] and [Touvron et al., 2023] has marked a paradigm shift in the field of artificial intelligence. These models have demonstrated unprecedented capabilities in a wide array of natural language understanding and generation tasks, including machine translation, content creation, summarization, and conversational AI. Their impact is transformative, driving innovation across both academic and industrial sectors. Despite their power, the practical deployment of LLMs is hindered by a significant computational bottleneck: the autoregressive generation process. By design, these models generate text sequentially, producing one token at a time, where each new token is conditioned on all previously generated ones. This inherently sequential nature underutilizes the parallel processing capabilities of modern hardware like GPUs, resulting in high inference latency. This bottleneck poses a substantial challenge for real-time applications and makes serving these models at scale a costly endeavor. Techniques like speculative decoding [Leviathan et al., 2023] have emerged to mitigate this issue by generating multiple draft tokens in parallel, but their effectiveness can vary significantly depending on the linguistic characteristics of the target language.

1.2 The Unique Landscape of Indian Languages

Indian languages, belonging to families such as Indo-Aryan and Dravidian, present a unique and complex landscape for Natural Language Processing. They are characterized by a high degree of morphological richness, often exhibiting agglutinative and inflectional properties. This means that words are frequently formed by appending a series of morphemes to a root to convey nuanced grammatical information, including case, gender, tense, and aspect. A single, morphologically complex word in a language like Kannada or Tamil can correspond to an entire phrase or multiple words in English.

These linguistic features pose distinct challenges for standard NLP models and their underlying tokenization methods:

- **Tokenization Fragmentation:** Subword tokenizers, which are standard for modern LLMs, may incorrectly split words into non-semantic fragments, breaking apart meaningful morphological units and obscuring the underlying grammar.
- **Data Sparsity:** The vast number of possible word forms for a given root leads to a long-tail distribution, where many valid forms appear rarely in training corpora, making it difficult for models to generalize.
- **Structural Complexity:** Flexible word order and the use of postpositions rather than prepositions challenge models that are heavily reliant on the fixed positional patterns prevalent in languages like English. As noted by [Karthika et al., 2025], these surface-level differences obscure a deeper, shared grammatical structure among Indian languages.

1.3 Problem Statement

The core problem addressed in this thesis is the dual challenge of inefficiency and linguistic inadequacy that arises when applying standard multi-token generation methods to the structural complexities of Indian languages. While

techniques like speculative decoding aim to accelerate inference, their naive application often fails to respect the linguistic integrity of these languages. A drafter model unaware of morphological boundaries may propose incoherent sequences by splitting semantically indivisible word groups: for instance, separating a noun from its case marker. This not only degrades the quality of the generated text, leading to ungrammatical and nonsensical outputs, but can also reduce the acceptance rate of drafted tokens, thereby diminishing the potential speedup.

1.4 Proposed Solution: A Linguistically-Informed Approach

To address this challenge, this thesis proposes a novel, linguistically-informed approach to speculative decoding. The core idea is to integrate deep linguistic knowledge, derived from the Samanvaya framework [Karthika et al., 2025], directly into the drafting process. Samanvaya provides a unified system for analyzing Indian languages by identifying "word groups" as the fundamental, indivisible semantic units of a sentence.

Our solution leverages a Suffix Automaton (SAM) as the engine for the drafter model. However, unlike a standard SAM, ours is constrained during its construction and draft generation phases to respect the word-group boundaries defined by Samanvaya. This ensures that the drafter can only propose multi-token sequences that are linguistically valid, thereby improving both the quality of the generated text and the efficiency of the speculative decoding cycle.

1.5 Key Contributions

- Design of a Linguistically-Constrained Suffix Automaton (LC-SAM) that is explicitly aware of semantic boundaries, ensuring that it generates drafts that are structurally and morphologically coherent.

- Novel application of the Samanvaya framework’s ”word groups” as necessary conditions within a multi-token prediction pipeline, bridging the gap between traditional linguistics and modern generative models for Indian languages.
- Development of an efficient Verifier-Drafter model where the LC-SAM acts as a specialized, high-speed drafter, tailored to the unique grammar of Indian languages, to work in concert with a general-purpose LLM verifier.

Chapter 2

Background and Literature Review

This chapter provides a comprehensive review of the foundational concepts and prior research that underpin this thesis. We begin by discussing the broader context of accelerating language model inference before focusing on the specific paradigm of speculative decoding. We then review two state-of-the-art speculative decoding frameworks, EAGLE and SAM-Decoding, which represent model-based and retrieval-based approaches, respectively. A detailed examination of the Suffix Automaton data structure, which is central to our work, is provided. Finally, we establish the linguistic foundation for our research by reviewing the Samanvaya framework and its principle of semantically cohesive word grouping for Indian languages.

2.1 Speculative Decoding: The Drafter-Verifier Paradigm

The high latency of autoregressive generation in Large Language Models (LLMs) has motivated the development of various acceleration techniques, including knowledge distillation, quantization, and pruning. Among these, speculative decoding [Leviathan et al., 2023] has emerged as a particularly effective, lossless method. The core principle of this paradigm is to decouple

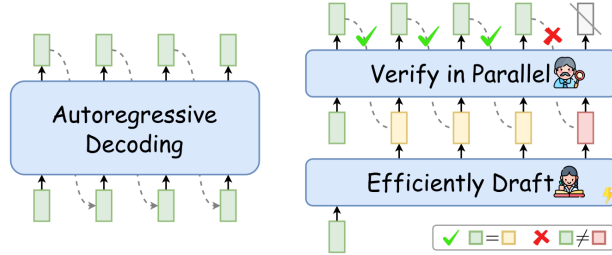


Figure 2.1: Autoregressive decoding(left) vs speculative decoding(right)

the token generation process into two distinct stages, executed by two different models: a "drafter" and a "verifier."

The **drafter** is a smaller, faster model that speculatively generates a sequence of candidate tokens (a "draft"). The **verifier** is the original, large, and more accurate LLM. The verifier takes the proposed draft and, in a single forward pass, evaluates the probability of each token in the sequence. The draft tokens are then accepted or rejected based on a statistical comparison between the drafter's and verifier's probability distributions. This parallel verification allows the model to accept multiple tokens at once, effectively reducing the number of required sequential generation steps and thereby accelerating the overall inference process(Figure 2.1).

2.2 State-of-the-Art in Speculative Decoding

EAGLE (Extrapolation Algorithm for Greater Language-model Efficiency) builds upon the idea of using auxiliary heads but introduces two key distinctions: feature prediction and explicit handling of sampling uncertainty [Li et al., 2025].

Unlike MEDUSA heads which predict token probabilities directly, EAGLE's "Autoregression Head" predicts the features (the hidden states from the second-to-last layer of the backbone LLM) for multiple future steps. The original, unmodified LM head of the target LLM is then used to generate the

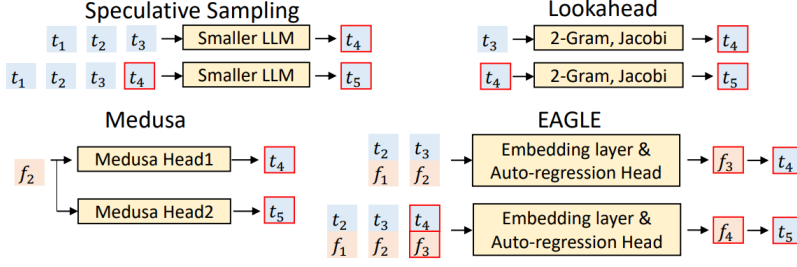


Figure 2.2: A comparison of the methods for drafting the fourth and fifth tokens, t_4 and t_5 .

token probability distributions from these predicted features. The rationale is that feature sequences might exhibit more regularity or be easier to predict autoregressively than token sequences (Figure 2.2).

A key challenge in predicting future steps is the uncertainty introduced by sampling at the current step. The feature at step $t+1$ depends not only on the features up to t but also on the specific token sampled or chosen at step $t+1$. EAGLE addresses this by incorporating the token generated/accepted in the previous verification step (t') as an additional input when predicting the feature for the next step ($t'+1$). The EAGLE Autoregression Head takes both the feature sequence and a token sequence advanced by one time step as input.

Predicting the next feature is framed as a regression task, for which the Smooth L1 loss is used (Figure 2.3):

$$\mathcal{L}_{\text{reg}} = \text{SmoothL1}(\mathbf{f}_{i+1}, \text{DraftModel}(T_{2:i+1}, \mathbf{F}_{1:i})).$$

This feature prediction serves as an intermediate objective for the draft model, whose ultimate goal is to generate the next token in a sequence. Therefore, a classification loss to guide the model toward token-level prediction is also applied:

$$\mathbf{p}_{i+2} = \text{Softmax}(\text{LMHead}(\mathbf{f}_{i+1})), \quad \hat{\mathbf{p}}_{i+2} = \text{Softmax}(\text{LMHead}(\hat{\mathbf{f}}_{i+1})),$$

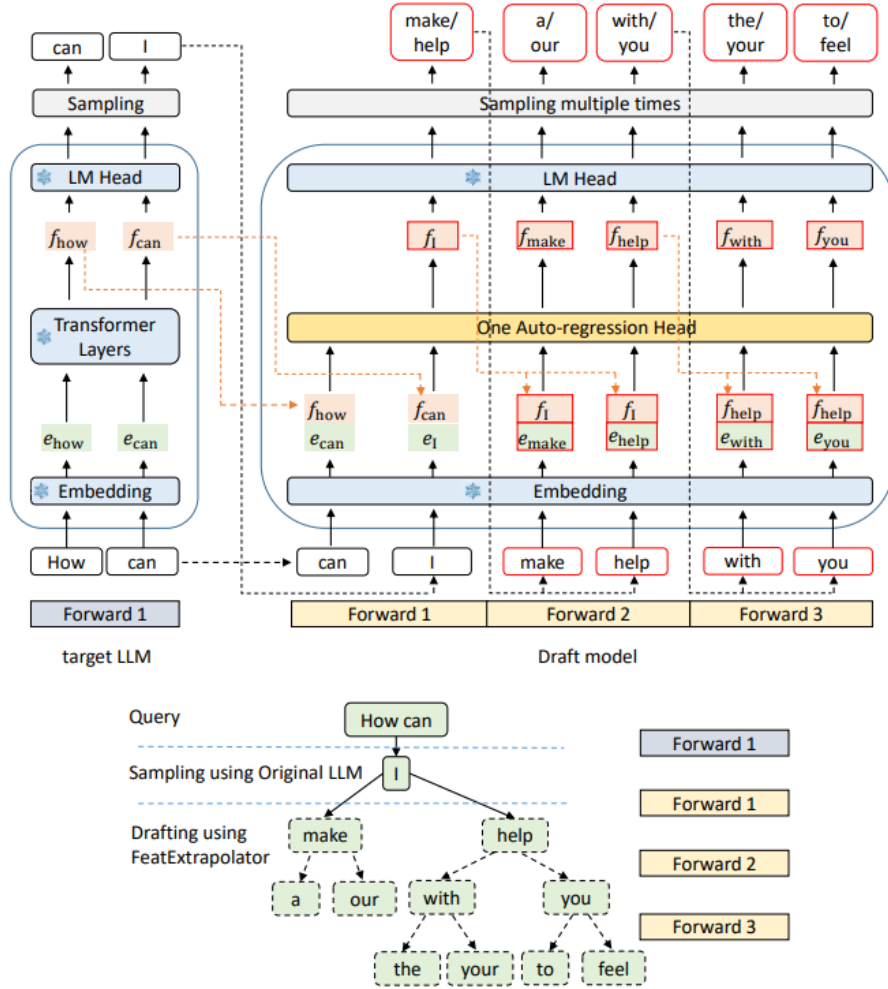


Figure 2.3: Pipeline of EAGLE. The upper section illustrates the computational process, while the lower section displays the corresponding generation results for each step.

$$\mathcal{L}_{\text{cls}} = \text{CrossEntropy}(\mathbf{p}_{i+2}, \hat{\mathbf{p}}_{i+2}).$$

The final loss used to train the Autoregression Head is a weighted sum of the regression and classification losses:

$$\mathcal{L} = \mathcal{L}_{\text{reg}} + w_{\text{cls}} \mathcal{L}_{\text{cls}}$$

Similar to MEDUSA, EAGLE generates a tree of candidate feature sequences using its Autoregression Head. It then uses the original LM head to get token distributions for all nodes in the tree. Tree attention is employed during the verification phase in the target LLM to efficiently process all candidates in parallel. EAGLE typically uses the standard rejection sampling or greedy acceptance mechanisms derived from speculative sampling to ensure lossless generation (Algorithm 1).

Algorithm 1 Multi-round Speculative Sampling

Input: Target distribution p , samples t_i and proposal distributions $\hat{p}_i$ for

$i = 1$ to k , where $t_i \sim \hat{p}_i$

Output: Sample $x \sim p$

$i \leftarrow 1$

for $i \leq k$ **do**

$r \leftarrow \mathcal{U}(0, 1)$

if $r < \frac{p(t_i)}{\hat{p}_i(t_i)}$ **then**

return t_i

end

$p \leftarrow \text{norm}(\max(0, p(t_i) - \hat{p}_i(t_i)))$

$i \leftarrow i + 1$

end

Sample $t \sim p$

return t

2.3 The Suffix Automaton: An Engine for Pattern Retrieval

In contrast to model-based methods, retrieval-based drafting generates candidate sequences by retrieving them from existing text sources. The SAM-

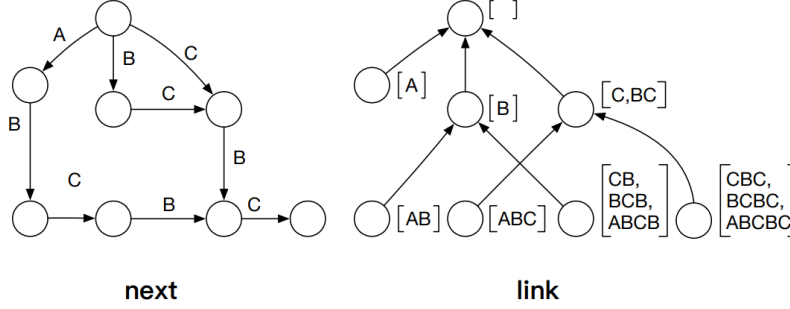


Figure 2.4: The suffix automaton corresponding to the string “ABCBC”.

Decoding framework [Hu et al., 2024] is a state-of-the-art implementation of this approach that utilizes a Suffix Automaton (SAM) for highly efficient and accurate draft generation.

SAM-Decoding leverages two retrieval sources: a large, general-purpose text corpus (indexed by a Static SAM) and the text sequence that has been dynamically generated so far in the current query (indexed by a Dynamic SAM). At each step, the framework finds the exact longest suffix of the current text that matches a pattern within its indexed sources. It then retrieves the tokens that followed that pattern as the draft. This method achieves an impressive average time complexity of $O(1)$ for both updating the automaton and retrieving a suffix, making it a lightweight and highly efficient drafter. This thesis builds directly upon the principles of SAM-Decoding, adapting its core machinery to incorporate linguistic constraints.

The Suffix Automaton is a powerful data structure that serves as the core engine for SAM-Decoding. A SAM is a minimal deterministic finite automaton (DFA) that recognizes all substrings of a given string. Its efficiency in both construction and lookup makes it ideal for pattern retrieval tasks.

A SAM is composed of states and transitions(Figure 2.4). Each state represents a set of substrings. The key components of a state, typically represented in a SAMState object, are:

- **next: dict[int, int]:** A dictionary of transitions. A transition from state u to v on token t means that if you append t to a substring ending at u , the new substring will end at v .

- **link: int:** The suffix link. The link from a state representing substring S points to the state representing the longest proper suffix of S . Suffix links provide a crucial "fallback" path for finding
- **length: int:** The length of the longest substring represented by the state.
- **min_endpos: int:** The end position of the first occurrence of the longest substring in the input text. This is vital for *gen_draft*, as it tells the model where to retrieve the continuation from.

A Static SAM is built offline from a large training corpus and remains unchanged during inference. The construction is linear in time with respect to the size of the corpus. The core of the construction process is the *add_state* method, which is executed for each token in the corpus. This algorithm performs the following key steps:

- **Create a New State:** A new state *cur* is created to represent the entire string processed so far.
- **Add Transitions:** Starting from the previous final state, the algorithm traverses backwards using suffix links, adding transitions from each state to the new state *cur* on the current token.
- **Set Suffix Link:** The algorithm then determines the appropriate suffix link for *cur*.
- **Handle State Cloning:** In cases where a new token extends a suffix in a way that violates the automaton's properties, an existing state is "cloned." This split creates two states: one for the shorter, shared suffix and another for the newly extended, longer string. This mechanism is critical for keeping the automaton minimal in size.

While a Static SAM captures general language patterns, a Dynamic SAM is designed to capture patterns specific to the current generation context. As detailed in the "Dynamic SAM Walkthrough," its key characteristics are: Online Construction: It starts empty at the beginning of each query.

- **Adaptive Growth:** For every token accepted by the verifier LLM, the Dynamic SAM is extended using the same *add_state* logic as the Static SAM. This allows it to learn repetitions and phrasing unique to the ongoing conversation.

Query-Specific Drafting: It provides drafts based only on the text seen in the current session.

Reset on New Query: The entire structure is wiped clean when a new query begins, preventing context from leaking between independent conversations.

This dual-automaton approach allows SAM-Decoding to leverage both long-term, general patterns and short-term, query-specific patterns for draft generation (Figure 2.5).

2.4 The Principle of "Semantically Cohesive Word Grouping"

While the engineering of speculative decoding has advanced rapidly, its application to diverse languages requires a corresponding depth of linguistic understanding. For Indian languages, the **Samanvaya** framework provides a critical foundation for re-evaluating the fundamental units of language processing [Karthika et al., 2025].

The central thesis of the Samanvaya framework is that the fundamental unit of meaning and syntactic function in many Indian languages is not the whitespace-separated word, but rather a multi-word **Local Word Group (LWG)**. These languages are often inflectional and agglutinative, meaning grammatical information (like case, tense, or aspect) is expressed through suffixes or by attaching separate post-positions and auxiliary verbs (Figure 2.6).

For example, the Hindi phrase **राम ने** (Rāma nē, "Ram did") functions as a single semantic unit (a noun with a case marker), even though it is two typographic words. Similarly, **जा रहा है** (jā rahā hai, "is going") is a single verb

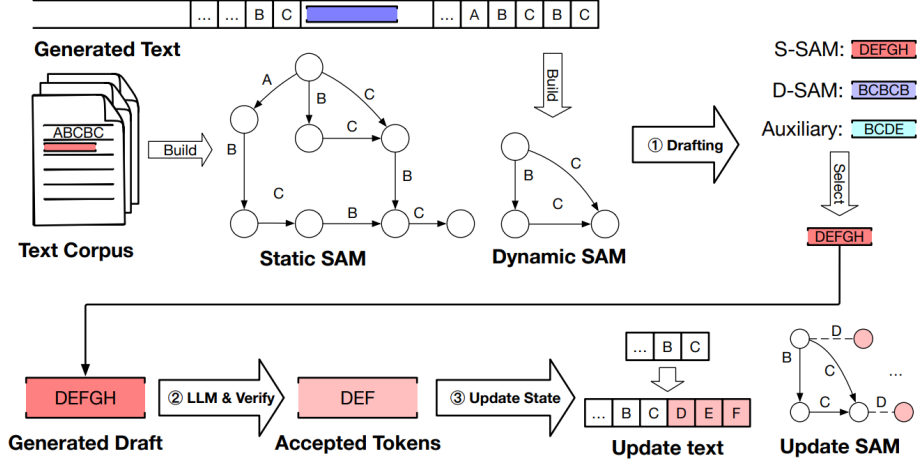


Figure 2.5: Overview of SAM-Decoding’s workflow. In each round of generation, the suffix automaton matches the suffixes of the generating text and retrieves the draft from the text corpus and the generated text respectively according to the matching position. Our method can be combined with an auxiliary SD algorithm (Auxiliary) to deal with the scenarios where the retrieval is not applicable. We select the best draft from the three candidate drafts based on the match length, and then the drafts are verified by the LLM for accepted tokens. Using these accepted tokens, we finally extend the dynamic SAM and generate text for the next round of generation.

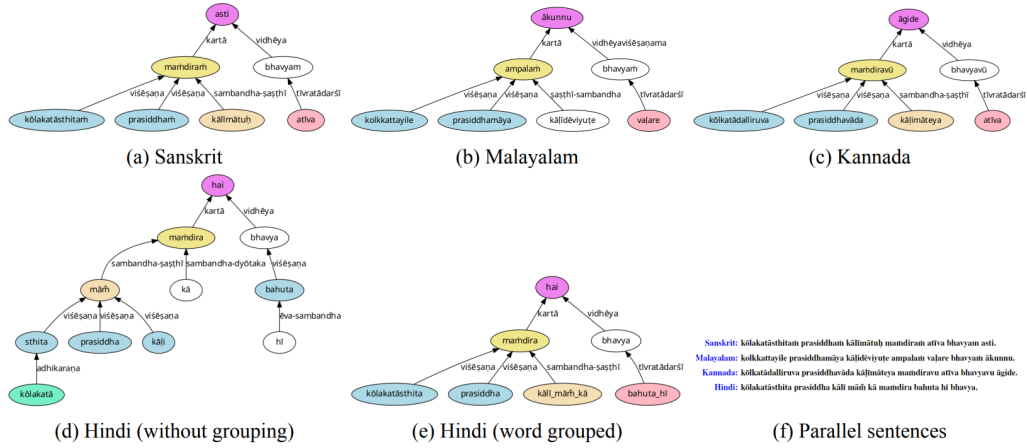


Figure 2.6: Dependency Parse Trees.

phrase unit. The Samanvaya framework argues that treating these LWGs as indivisible units helps to unify the syntactic structures of parallel sentences across different Indian languages and provides a more accurate representation of their morphology.

The principle of word grouping has profound implications for computational models. Standard tokenization schemes and generation models, which treat whitespace as a primary delimiter, inadvertently break these semantic units. This fragmentation can lead to several problems:

- **Loss of Meaning:** The model may fail to capture the holistic meaning of the word group.
- **Ungrammatical Generation:** A generation model, especially a drafter in speculative decoding, might propose a partial word group as a complete sequence, leading to incoherent and grammatically incorrect outputs.
- **Inefficient Learning:** The model is forced to learn long-distance dependencies between fragments of a semantic unit, a task that would be much simpler if the unit were treated as a single token.

This linguistic insight provides the central motivation for this thesis. If the fundamental unit of meaning is the word group, then a generation model, particularly a high-speed drafter, must be designed to respect these boundaries. This leads directly to our proposed methodology of creating a Word-Group-Aware Suffix Automaton.

Chapter 3

Data Preparation: Creating a Linguistically-Informed Corpus

The success of a retrieval-based speculative decoding system like SAM-Decoding is fundamentally dependent on the quality and structure of the corpus used to build its static automaton. As established in the previous chapter, standard tokenization schemes often fail to capture the semantic structure of Indian languages. This chapter details the methodology for preparing a specialized, linguistically-informed corpus that explicitly encodes the word group boundaries identified by the Samanvaya framework. This boundary-aware data is the cornerstone upon which our proposed Linguistically-Constrained Suffix Automaton is built.

3.1 The Need for Boundary-Aware Data

A standard Static Suffix Automaton, as described in Chapter 2, is built from a flat sequence of tokens. While it can efficiently find and retrieve any substring from this sequence, it is inherently agnostic to any higher-level linguistic structure. It cannot distinguish between a sequence of tokens that forms a complete, meaningful semantic unit and a sequence that is merely an arbitrary fragment.

For Indian languages, this limitation is particularly acute. For example,

a SAM built on the token sequence [कर, ना, टक, ने, भा, रती, य] might identify [टक, ने, भा] as a frequent substring and propose it as a draft. However, this sequence is linguistically nonsensical, as it incorrectly joins the end of one word group (कर्नाटक_ने) with the beginning of another (भारतीय). The standard SAM lacks the necessary information to know that the boundary after ने is a "hard" semantic stop.

To overcome this, we require a new data format that enriches the raw token sequence with explicit metadata marking the end of each Local Word Group (LWG). This allows us to build a modified automaton that can leverage this information to generate drafts that are not only statistically likely but also linguistically well-formed.

3.2 Leveraging the Samanvaya Framework for Annotation

To annotate our corpus with word group boundaries, we leverage the rule-based methodology proposed in the "Semantically Cohesive Word Grouping in Indian Languages" paper [Karthika et al., 2025]. The successful construction of a Word-Group-Aware Suffix Automaton (WGA-SAM) is contingent upon a training corpus that explicitly encodes the semantic boundaries defined by the Samanvaya framework. Since large, pre-annotated corpora are not readily available, we have developed a deterministic, rule-based methodology to process raw Hindi text and annotate it with word-group boundaries. This process transforms a sequence of whitespace-separated tokens into a sequence of semantically cohesive units, providing the necessary input for building our linguistically-constrained drafter model. This approach provides a systematic way to identify LWGs based on syntactic and morphological principles.

3.2.1 The Rule-Based Word Grouping Process

The annotation process is implemented as a multi-stage pipeline where a series of heuristic rules are applied sequentially to a given sentence. The ordering of these rules is crucial, as some grouping decisions depend on others. The pipeline is designed to first handle larger, fixed phrases and complex verb chains, followed by more granular rules for right- and left-attaching particles and postpositions.

The overall workflow for a given sentence is as follows:

1. **Tokenization:** The input sentence is split into a list of tokens based on whitespace.
2. **Stage 1: Phrase and Verb Grouping:** The pipeline first identifies and groups fixed multi-word expressions and then iteratively constructs complex verb phrases. This ensures that these larger, often idiomatic or grammatically fixed units are treated as single entities before more general attachment rules are applied.
3. **Stage 2: Right-Attachment:** A specific set of rules is applied to attach certain modifiers, like negation particles and numerals, to the token on their right.
4. **Stage 3: Left-Attachment:** The most common rule is then applied, attaching postpositions and various particles to the token on their left. This step consolidates the majority of inflectional word groups.

The output of this pipeline is a sequence of grouped tokens, which are then formatted to clearly delineate the word-group boundaries for the SAM construction process.

3.2.2 Handling Inflectional, Derivational, and Compound Word Groups

The rule set is designed to capture the various types of LWGs identified in the Samanvaya framework. For instance:

- **Inflectional Unity:** A rule identifies a noun (NN or NNP) followed by a post-position (PSP). The post-position is marked as the end of the group. Example: In हाथ से (hāth se, "with hand"), से (se) is a PSP dependent on हाथ (hāth), triggering the grouping हाथ_से.
- **Derivational Unity:** A rule identifies a main verb (VM) followed by one or more auxiliary verbs (VAUX). The final auxiliary verb is marked as the end of the group. Example: In जा रहा है (jā rahā hai, "is going"), the auxiliaries रहा (rahā) and है (hai) are grouped with the main verb जा (jā) to form जा_रहा_है.
- **Compound Word Groups:** Rules based on NNPC (noun-noun compound) dependency relations or specific named entity patterns are used to group multi-word compounds and named entities.

By applying these rules systematically, we transform an unannotated text corpus into a dataset where the boundaries of these crucial semantic units are explicitly marked.

Each stage in the pipeline is designed to handle specific types of word groups as identified by the Samanvaya framework. The implementation details of each stage are described below.

This initial step handles compound phrases and common multi-word expressions that function as a single semantic unit. A predefined list of such phrases is used to scan the token sequence. To ensure correctness, the matching process prioritizes longer phrases over shorter ones. For instance, the three-word phrase "हाल ही में" (hāl hī meṃ, "recently") is matched before the two-word phrase "ही में" (hī meṃ, "in this very").

Examples of such fixed phrases include:

- **Adverbial Phrases:** ["हाल", "ही", "में"] → हाल_ही_में
- **Compound Postpositions:** ["के", "लिए"] → के_लिए (ke lie, "for"), ["के", "दौरान"] → के_दौरान (ke daurān, "during")
- **Common Expressions:** ["मुकर", "जाएं"] → मुकर_जाएं (mukar jāeṃ, "to back out")

This rule is designed to handle derivational word groups, specifically the complex verb chains common in Hindi that consist of a main verb stem and one or more auxiliary verbs. The process is iterative to handle verb groups of varying lengths (e.g., कर_दी_थी).

The logic identifies verb stems based on their characteristic endings and combines them with a known set of subsequent auxiliaries:

- A verb ending in -ा (A_ENDING) is grouped with a following auxiliary from a specific set (e.g., गया, जाता, है).

– Example: ["जाता", "है"] → जाता_है

- A verb ending in -ी (EE_ENDING) is grouped with its corresponding set of auxiliaries (e.g., गई, जाती, है).

– Example: ["जाती", "है"] → जाती_है

- Special cases are also handled, such as verbs followed by चाहिए (cāhie, "should") and expressions like लगता है (lagtā hai, "it seems").

After handling larger phrases, a specific rule is applied for modifiers that grammatically attach to the word on their right. This is less common in Hindi but applies to critical cases:

- **Negation:** The negation particle नहीं (*nahīṁ*, "not") is grouped with the following token.

– Example: ["नहीं", "जाएगा"] → नहीं_जाएगा

- **Numerals:** Numeric tokens are grouped with the noun they quantify.

– Example: ["5", "लोग"] → 5_लोग

This final and most frequently applied stage handles the vast majority of inflectional word groups. It groups postpositions and various emphatic or conjunctive particles with the preceding token (typically a noun or pronoun). A comprehensive set of such tokens is defined for left-attachment.

This rule is responsible for creating many of the core semantic units in a sentence:

- **Postpositions (Case Markers):** से, में, का, के, की, को, पर, ने are attached to the preceding word.
 - Example: ["राम", "ने"] → राम_ने
 - Example: ["घर", "में"] → घर_में
- **Particles:** Emphatic particles like ही ($hī$, “only”) and भी ($bhī$, “also”) are attached.
 - Example: ["उसने", "ही"] → उसने_ही
- **Compound Postpositions:** This rule also handles the output of Stage 1(a), attaching pre-grouped compound postpositions.
 - Example: ["घर", "के लिए"] → घर_के_लिए

After the cascaded application of these rules, a sentence like

राम ने भी घर के लिए 5 आम नहीं खरीदे

is transformed into the grouped string

राम_ने_भी घर_के_लिए 5_आम नहीं_खरीदे.

This output is then segmented to produce the final boundary-aware format ["राम##ने##भी", "घर##के##लिए", "5##आम", "नहीं##खरीदे"] which serves as the direct input for constructing the Word-Group-Aware Suffix Automaton.

3.3 Designing the Data Structure

To feed this enriched information into our modified Suffix Automaton, we designed a specific data structure.

3.3.1 Representing Tokens and Boundaries

For each sentence, the final representation is a JSON object containing two parallel lists:

1. **"tokens"**: A list of integer token IDs, produced by the target model's tokenizer (e.g., Airavata's tokenizer). Standard tokenization is still used to get the base units, as the SAM operates on these integer IDs.
2. **"word_group_boundaries"**: A list of booleans of the same length as the "tokens" list. A value of *True* at index i indicates that the token $tokens[i]$ is the final token of a Local Word Group. A value of *False* indicates that it is not.

This parallel structure is simple, efficient to store and process, and directly provides the necessary boundary information for each token in the sequence.

3.3.2 Example of an Annotated Sentence

Let's consider the Hindi phrase "कर्नाटक ने भारतीय शास्त्रीय संगीत के" (Karnāṭak ne bhāratīya śāstrīya saṅgīt ke, "Karnataka's Indian classical music's...").

- **Linguistic Grouping (Samanvaya)**: The phrase contains two primary LWGs: कर्नाटक_ने (Karnāṭak_ne) and भारतीय_शास्त्रीय_संगीत_के (bhāratīya_śāstrīya_saṅgīt_ke). (3.1).
- **Step 1: Tokenization**: The phrase is tokenized by the LLM's tokenizer. For illustration, let's assume a character-level tokenization for simplicity (in reality it would be subwords):
Text: कर्नाटक ने भारतीय शास्त्रीय संगीत के
Tokens: ['क', 'र्', 'ना', 'ट', 'क', ' ', 'ने', ' ', ' ', 'भा', 'र', 'त्ती', 'य', ' ', ' ', 'शास्', 'त्री', 'य', ' ', ' ', 'सं', 'गी', 'त', ' ', ' ', 'के']
- **Step 2: Boundary Annotation**: Based on the grouping rules, the boundaries are marked at the end of ने and के.

Text:	कर्नाटक##ने				भारतीय शास्त्रीय संगीत##के									
	Group 1				Group 2									
Tokens:	[कर्,	ना,	टक,	ने,	भा,	रती,	य,	शास्,	त्री,	य,	सं,	गी,	त,	के]
Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13
Boundary:	[F	F	F	T	F	F	F	F	F	F	F	F	F	T]
				↑										↑
				End of										End of
				Group 1										Group 2

Figure 3.1: Dependency Parse Trees.

- **Step 3: Final Data Structure:**

Listing 3.1: token boundaries

```
{
  "tokens": [291, 351, 292, 353, 259, 361, 293, ...],
  "word_group_boundaries": [
    false, false, false, true, // कर्नाटक_ने
    false, false, false, // भारतीय
    false, false, false, // शास्त्रीय
    false, false, false, // संगीत
    true // के
  ]
}
```

3.4 Corpus Curation and Preprocessing Pipeline

The complete data pipeline to create the final boundary-annotated corpus is as follows:

1. **Source Corpus Selection:** We begin with a corpus of high-quality Hindi text. For this work, we used the Hindi portion from the WikiChat subset of Indic-Align Instruct dataset [Khan et al., 2024], ensuring the data is relevant to our target model’s domain.

2. **Sentence Segmentation:** The corpus is segmented into individual sentences.
3. **LWG Annotation** Each sentence is processed through the rule-based pipeline described in Section 3.2 to identify LWG boundaries. This produces sentences where multi-word groups are joined by an underscore (e.g., राम_ने).
4. **Tokenization and Boundary Mapping:** Each annotated sentence is processed with the target model’s (Airavata’s) tokenizer. As the sentence is tokenized, we track the original word boundaries. When the tokenizer processes a word that was marked as the end of an LWG, the boolean flag for the final token corresponding to that word is set to *True*.
5. **Final Corpus Generation:** The resulting lists of token IDs and boolean boundary flags are serialized into a JSONL file (one JSON object per line, representing one sentence), forming the corpus.

This final corpus is now ready to be used as input for building the Linguistically-Constrained Static Suffix Automaton, which is the subject of the next chapter.

Chapter 4

Methodology: The Word-Group-Aware SAM (WGA-SAM) Framework

This chapter presents the core technical contribution of this thesis: the Word-Group-Aware Suffix Automaton (WGA-SAM) framework. Building upon the foundations of SAM-Decoding and the linguistic principles of the Samanvaya framework, we introduce a novel modification to the Suffix Automaton data structure and its associated algorithms. This modification enables the speculative drafting process to respect the semantic boundaries of Local Word Groups (LWGs), aiming to produce drafts that are not only statistically likely but also linguistically well-formed.

4.1 Architectural Overview

The proposed WGA-SAM framework integrates seamlessly into the standard drafter-verifier paradigm of speculative decoding. The key innovation is the replacement of the standard Static SAM component within the draft model with our new Word-Group-Aware SAM. The overall architecture remains a hybrid system, capable of falling back to other drafting methods (like a model-based tree drafter or a standard Dynamic SAM) when a high-quality,

linguistically-constrained retrieval is not possible.

The workflow is as follows: during a generation step, the *WordGroupAwareDraftModel* first attempts to find a long suffix match using the Dynamic SAM. If that fails or the match is short, it queries the WGA-SAM. The WGA-SAM, now equipped with knowledge of word group boundaries, retrieves a draft that is guaranteed not to cross a semantic unit boundary. This linguistically-sound draft is then passed to the verifier LLM for parallel verification and acceptance.

Algorithm 2 Word-Group-Aware Speculative Decoding Workflow for Hindi

- 1: **Input:** processed_nios.jsonl
 - 2: **Output:** Generated Hindi text respecting word group boundaries
 - 3: Load JSONL:
 - 4: tokens $\leftarrow$ token IDs
 - 5: word_group_boundaries $\leftarrow$ boolean flags
 - 6: **Stage 1: Build WordGroupAwareSAM**
 - 7: Initialize WordGroupAwareSAM object
 - 8: Store states, input_ids, word_boundaries
 - 9: Define `gen_draft(index, start_token)`:
 - 10: Find match position
 - 11: Generate token sequence
 - 12: Check `word_boundaries[pos]` $\rightarrow$ Stop if boundary=True
 - 13: **Stage 2: Initialize Draft Model**
 - 14: Initialize WordGroupAwareDraftModel:
 - 15: sam_static $\leftarrow$ WordGroupAwareSAM
 - 16: sam_dyn $\leftarrow$ DynSAM
 - 17: tree_model $\leftarrow$ TreeModel
 - 18: Define `lookup(start_token)`:
 - 19: Check dynamic SAM
 - 20: Check static SAM (respecting word boundaries)
 - 21: Choose best draft
 - 22: Return token sequence or tree
 - 23: **Stage 3: Perform Speculative Decoding**
 - 24: Pass input tokens to SamdModel
 - 25: Use WordGroupAwareDraftModel for draft generation
 - 26: Verify drafts respecting word group boundaries
 - 27: Generate final Hindi text
 - 28: **Return:** Generated Hindi text
-

4.2 Core Innovation: Modifying the Suffix Automaton

The central innovation lies in augmenting the Suffix Automaton’s data structure and construction process to be aware of word group boundaries.

4.2.1 Extending the Data Structure with Boundary Information

To enable boundary-aware drafting, we modify the `StaticSAM` class to store the boundary information alongside the token sequence. A new attribute is introduced:

- **word_boundaries: List[bool]:** This list is loaded from the preprocessed corpus (as described in Chapter 3) and has the same length as the main `input_ids` list. Each boolean value at index `i` corresponds to the token at `input_ids[i]`, indicating whether that token is the end of an LWG.

The `SAMState` dataclass itself does not require modification. The boundary information is not stored per-state but is instead associated with the global `input_ids` sequence from which drafts are retrieved. The logic for leveraging this information is embedded within the draft generation algorithm.

4.2.2 The Boundary-Aware build Algorithm

The offline construction process for the WGA-SAM is adapted to handle the new data format. The build method is modified to load not just the token sequences but also the corresponding boundary lists from the processed `_nios.jsonl` file.

During the `add_batch_tokens` loop, as each sentence’s token list is appended to the global `self.input_ids`, its corresponding boundary list is simultaneously appended to the global `self.word_boundaries`. The core `add_state`

algorithm for building the automaton’s structure (states, transitions, links) remains unchanged, as it operates only on the sequence of token IDs. The boundary information is passively loaded and stored in parallel, ready to be used during the inference-time draft generation phase.

4.2.3 Aggregating Multiple End-Positions for Draft Diversity

A key limitation of the standard SAM-Decoding approach is its reliance on a single *min_endpos* value. This means that for any given matched context, the drafter can only propose one continuation, creating a single point of failure. If this single draft is incorrect, the performance gain of the speculative step is minimal. To overcome this, we enhance the WGA-SAM to store and utilize multiple occurrence locations, enabling the generation of a diverse set of draft candidates for each decoding step.

This is achieved through a two-stage process: recording occurrences during construction and aggregating them in a post-build finalization step.

1. **Terminal Occurrence Logging during Construction:** The build algorithm is augmented with a lightweight logging mechanism. As the automaton is constructed, each time a new state is created representing the full sequence processed so far, we record a tuple of (*state_index*, *endpos*). This tuple captures the fact that the automaton terminated in *state_index* at position *endpos* in the training corpus. This logging adds only O(1) overhead per token and creates a comprehensive record of all terminal occurrences throughout the corpus.
2. **Post-Build Finalization and Candidate Propagation:** After the full automaton has been constructed, a *finalize_endpos_candidates* procedure is executed. This step aggregates the logged occurrences to equip each state with a list of potential draft starting points. The process works as follows:
 - **Initialization:** A list (or "bucket") is created for each state in the automaton to hold its *endpos* candidates.

- **Bottom-Up Propagation:** The algorithm processes states in order of decreasing string length. For each state, it first normalizes its own list of directly recorded *endpos* occurrences (by sorting and deduplicating). It then propagates this list to the bucket of its parent state, as determined by the suffix link.
- **Aggregation and Truncation:** As this process continues, each state effectively accumulates all the *endpos* occurrences from itself and from all longer substrings that share it as a suffix. At each state, this aggregated list is truncated to a predefined limit, k .

This bottom-up propagation along suffix links ensures that states representing shorter, more common substrings (like "is a") inherit the rich contextual occurrences of longer, more specific substrings (like "Paris is a" and "Delhi is a"). By storing the top k end-positions in a new field, *state.endpos_candidates*, we provide the drafter with multiple distinct contexts from which to generate drafts. This entire finalization step has a practical complexity of $O(|States| * k)$, making it highly efficient.

It is important to note the heuristic used for selecting the top- k candidates. The current implementation prioritizes the earliest occurrences in the corpus by sorting *endpos* values in ascending order. While this is a configurable detail, it may favor continuations that appeared earlier in the training data.

4.3 Advanced Draft Generation: From Single to Multiple Candidates

The most significant modification is to the draft generation logic. We introduce a new *gen_draft* algorithm that actively uses the stored boundary information to constrain the output.

4.3.1 The Modified *gen__draft* Algorithm

The standard *gen__draftmethod* in a SAM retrieves a fixed number of tokens (*n_predicts*) that follow a matched suffix. Our modified algorithm introduces a crucial check within the retrieval loop.

Algorithm Logic:

1. **Find Match Position:** After a suffix match is found, determine the *endpos* of its first occurrence in the *input_ids* corpus, as in the standard algorithm.
2. **Initialize Draft:** Start retrieving tokens from *pos = endpos + 1*.
3. **Iterative Token Retrieval with Boundary Check:**
 - Append the next token *input_ids[pos]* to the draft sequence.
 - Crucially, check the boundary flag *self.word_boundaries[pos]*.
 - If the flag is *True*, it signifies the end of an LWG. Terminate the draft generation immediately and break the loop.
 - If the flag is *False*, increment *pos* and continue to the next token, until the draft reaches a predefined maximum length or the end of the corpus.
4. **Return Constrained Draft:** Return the generated draft, which is now guaranteed to be a prefix of a longer sequence that ends at or before the next LWG boundary.

4.3.2 Generating Multiple Draft Candidates via Top-k End-Positions

With the *endpos_candidates* list available for each state, we introduce a new method, *gen__draft_candidates(index, start_token, k)*, which significantly enhances the drafting capability. Instead of retrieving a single continuation based on *min_endpos*, this method iterates through the *state.endpos_candidates* list stored at the matched index.

For each *endpos* in the list, it constructs a candidate draft by retrieving the token sequence starting from *endpos* + 1 in the source corpus. This allows the drafter to propose multiple, diverse continuations for the same input context, increasing the probability that at least one of the drafts will align with the verifier’s prediction.

Crucially, this multi-draft generation process remains fully constrained by our linguistic rules. Each candidate sequence, regardless of its source *endpos*, is processed by the boundary-aware logic described in section 4.3.1. The generation of each draft is terminated at the first encountered word-group boundary, ensuring that all proposed drafts are not only diverse but also individually well-formed and linguistically valid according to the Samanvaya framework.

4.4 The Complete Verifier-Drafter Workflow

4.4.1 The Foundational Single-Draft Workflow

The fundamental integration of the WGA-SAM as a drafter operates on a one-draft-per-step basis. This workflow serves as the baseline for our linguistically-informed approach and ensures that every proposed sequence adheres to the principles of the Samanvaya framework. A single generation cycle in this workflow proceeds as follows:

1. The *SamdModel* requests a draft from the *WordGroupAwareDraftModel*.
2. The lookup method is called. It first checks for a match in the query-specific *DynSAM*.
3. If no sufficient match is found in *DynSAM*, it checks for a match in the *WGA – SAM*.
4. If the *WGA – SAM* finds a match, it executes the modified *gen_draft* algorithm, retrieving a token sequence that stops at the next LWG boundary.

5. If no sufficient match is found in either SAM, the system falls back to a model-based drafter (e.g., EAGLE heads).
6. The chosen draft (from whichever source) is returned to the *SamdModel*.
7. The *SamdModel* performs parallel verification of the draft using the target LLM (Airavata).
8. The longest valid prefix is accepted using a lossless acceptance criterion.
9. The accepted tokens are used to update the *DynSAM* and the generation context for the next cycle.

While this single-draft approach guarantees the linguistic validity of every proposal, it is inherently limited by the fact that the verifier’s optimal continuation may stem from a different, less common context than the one identified by the *min_endpos* heuristic. If this single draft is rejected, the efficiency gain for that step is lost. This limitation motivates the development of a more robust, multi-candidate approach.

4.4.2 The Advanced Multi-Draft Workflow with Candidate Verification

To significantly increase the probability of a successful speculation at each step, we extend the foundational workflow to generate and verify multiple draft candidates in parallel. This advanced workflow leverages the *endpos_candidates* list, which is aggregated during the SAM’s finalization phase as described in Section 4.2.3, and the multi-draft generation logic from Section 4.3.2.

The complete verifier-drafter workflow in this advanced configuration is as follows:

1. **Context Matching:** The system takes the current sequence of accepted tokens and traverses the WGA-SAM to find the state that represents the longest matching suffix.

2. **Multi-Draft Generation:** The *gen_draft_candidates* method is invoked on the matched state. The WGA-SAM proposes **a set of k candidate drafts**, where each draft is a linguistically valid word group retrieved from a different occurrence in the corpus.
3. **Parallel Verification:** The Verifier LLM receives this batch of candidate sequences, which can be structured as a prefix tree (trie) to maximize computational efficiency. Using a mechanism like tree attention, the verifier evaluates all unique paths across the drafts in a single parallel forward pass, computing the true probability distributions for each token position across all candidates.
4. **Best-Path Acceptance and Correction:** The system identifies the optimal path among the verified candidates by comparing the drafted tokens with the verifier’s predictions. The longest sequence of consecutively matched tokens from the entire set of drafts is chosen as the output for this step. If no token across any of the k drafts is accepted, the system re-samples a single token based on the verifier’s distribution at the first position, ensuring the generation process remains lossless.

By generating multiple, diverse, and linguistically-sound drafts, this advanced workflow maximizes the chance of finding a continuation that aligns with the verifier’s prediction. This increases the average number of tokens accepted per verification step and provides a more substantial acceleration of the overall inference speed compared to the single-draft approach.

Chapter 5

Experimental Setup and Results

This chapter details the experimental framework designed to evaluate the performance of our proposed Word-Group-Aware SAM (WGA-SAM). We first describe the datasets used for building our models and for evaluation, followed by the specific model configurations and metrics. Finally, we present a comprehensive analysis of the results, comparing our linguistically-informed approach against established baselines and providing an in-depth, token-level breakdown of drafter performance.

5.1 Experimental Setup

To ensure a fair and reproducible comparison, all experiments were conducted within a standardized environment using carefully selected datasets and evaluation protocols.

The foundation of our linguistically-aware static drafter is a high-quality, domain-specific corpus. For this purpose, we utilized the Hindi portion of the **WikiChat dataset**, a subset of the larger **Indic-Instruct** suite [Khan et al., 2024].

The WikiChat dataset is composed of conversations between a user and an assistant that are grounded in context from Wikipedia articles. This makes

the data rich in factual, well-structured, and encyclopedic language. For our experiments, we used the Hindi subset, which contains approximately **19,000 conversational rows**. Each row represents a turn in a dialogue, providing a diverse set of sentences and phrasal constructs.

This dataset served two critical functions in our setup:

1. It was used as the source corpus for building both the baseline **Static SAM** and our proposed **WGA-Static SAM**. For the WGA-SAM, the corpus was pre-processed using the rules described in Chapter 3 to annotate word-group boundaries.
2. Crucially, this same dataset was also used to train the **EAGLE-2 tree decoding model** for the Airavata model. This ensures a fair comparison, as all drafter components (our SAM-based drafters and the EAGLE-2 drafter) were trained on or built from the same underlying data distribution.

To evaluate the performance of our framework, we used the MT-Bench-Hindi dataset, a high-quality benchmark for assessing the conversational and instruction-following capabilities of LLMs in Hindi [Kamath et al., 2025].

The dataset consists of **200 high-quality, multi-turn questions** covering eight categories, including reasoning, writing, and roleplay. It was created using a hybrid methodology that combines professional translation of the original English MT-Bench with from-scratch creation of culturally relevant questions, ensuring deep linguistic and contextual relevance for the Hindi language. Its focus on multi-turn dialogue makes it an ideal benchmark for evaluating the practical speedup and coherence of generative models in real-world scenarios.

Our evaluation compares the performance of three different model configurations, all running within the Spec-Bench framework [Xia et al., 2024] to standardize the evaluation process:

1. **Baseline (EAGLE-2):** The EAGLE-2 model serving as a standalone drafter for a verifier LLM.

2. **EAGLE-2 + SAM:** The original SAM-Decoding implementation, which combines the EAGLE-2 drafter with both a Static and a Dynamic SAM drafter.
3. **Our Proposal (EAGLE-2 + WordGroupAwareSAM):** Our proposed framework, which combines the EAGLE-2 drafter with our linguistically-constrained WGA-Static SAM and a standard Dynamic SAM.

The performance of each configuration was measured using the following key metrics:

- **Mean Accepted Tokens (Overall):** The average number of tokens accepted by the verifier per verification step. This is a direct measure of the drafters’ collective efficiency.
- **Tokens/s:** The total number of tokens generated divided by the total wall-clock time, measuring the end-to-end throughput of the system.
- **Speedup:** The ratio of the wall-clock time taken by our model compared to a standard autoregressive baseline, quantifying the overall inference acceleration.

5.2 Results and Analysis

We present the results in two parts. First, we analyze the overall comparative performance of our WGA-SAM framework against the baselines. Second, we provide an in-depth, token-level breakdown to understand the individual contributions of the static and dynamic drafter components.

The overall performance of the three configurations on the MT-Bench-Hindi dataset is summarized in Table 5.1. The results are presented for two different hyperparameter settings, controlling the preference for SAM-based drafts over EAGLE-2 drafts.

From the results, we observe that while both SAM-based methods significantly outperform the standalone EAGLE-2 baseline, our proposed WordGroupAwareSAM consistently shows a lower speedup and Mean Accepted

Table 5.1: Comparative Performance on MT-Bench-Hindi

Configuration	#MAT	Tokens/s	Speedup
Bias = 0, Threshold = 0			
EAGLE-2 (Baseline)	3.391	156.78	1.488×
EAGLE-2 + SAM	5.117	324.60	3.081×
EAGLE-2 + WordGroupAwareSAM	4.925	290.34	2.756×
Bias = 2, Threshold = 3			
EAGLE-2 (Baseline)	3.391	156.78	1.488×
EAGLE-2 + SAM	5.983	330.75	3.140×
EAGLE-2 + WordGroupAwareSAM	4.925	299.96	2.850×

Tokens (MAT) compared to the original, unconstrained SAM implementation. This counter-intuitive result can be attributed to a fundamental trade-off between draft length and linguistic precision.

The primary reasons for this performance difference are:

1. **Systematic Draft Truncation:** The core feature of WGA-SAM is its *gen_draft* algorithm, which is designed to terminate a draft at the end of a semantic word group. This frequently results in shorter, albeit more grammatically correct, drafts. In contrast, the unconstrained SAM generates drafts of a fixed, longer length, which can lead to more accepted tokens before a mismatch, even if the tail end of the draft is incorrect.
2. **Draft Length vs. Acceptance Rate Trade-off:** Speculative decoding benefits most from long accepted sequences. A longer draft from the standard SAM, even if only partially correct, may yield more accepted tokens than a shorter, fully correct draft from WGA-SAM. For example, if a 7-token draft has its first 4 tokens accepted, it contributes more to the overall speedup than a fully accepted 3-token draft.
3. **Limited Static Corpus Coverage:** As will be shown in the next section, the static drafter’s contribution is minimal due to the limited size of the WikiChat corpus. This forces the system to rely heavily on

the dynamic drafter, where the effect of draft truncation in WGA-SAM is more pronounced.

To better understand the behavior of each drafter component, we conducted a detailed token-level analysis. Table 5.2 shows the number of tokens drafted, accepted, and the corresponding acceptance rates for the Static and Dynamic components of both the standard SAM and our WGA-SAM.

Table 5.2: In-depth Token-Level Drafter Performance

Setting	Method	Drafter	Drafted Tokens	Accepted Tokens	Steps	Acceptance Rate	Mean Accepted Tokens (per method call)
Bias = 2 Threshold = 3	SAM	Static SAM	44534	1659	3181	3.73%	0.52
		Dynamic SAM	9236	5931	6594	64.25%	0.90
	WGA SAM	Static SAM	44618	1670	3187	3.74%	0.52
		Dynamic SAM	99974	57178	7141	57.19%	0.81
Bias = 0 Threshold = 0	SAM	Static SAM	233604	5956	16686	2.55%	0.36
		Dynamic SAM	195874	65174	13991	33.27%	0.47
	WGA SAM	Static SAM	233674	5952	16691	2.55%	0.36
		Dynamic SAM	205072	64460	14648	31.43%	0.44

This detailed breakdown reveals several key insights:

- **The Dominance of Dynamic SAM:** In all configurations, the Dynamic SAM is the primary contributor to accepted tokens. Its acceptance rate is exceptionally high (e.g., 64.25% in the Bias 2 setting for the standard SAM), demonstrating its effectiveness at capturing query-specific repetitions.
- **Minimal Contribution of Static SAM:** Conversely, the Static SAM exhibits a very low acceptance rate (2-4%) across all experiments. This confirms our hypothesis that its effectiveness is severely limited by the pattern coverage of the 19k-row WikiChat corpus. It is simply unable

to find relevant, high-quality matches frequently enough to make a significant impact.

- **Impact of Word-Group Awareness:** The WGA-SAM’s dynamic component drafts significantly more tokens than the standard SAM’s (e.g., 99k vs. 9k in the Bias 2 setting). This is because its shorter, boundary-aware drafts lead to more frequent calls to the drafter. However, its mean accepted tokens per call is slightly lower (0.81 vs. 0.90), a direct consequence of the draft truncation effect. While each draft is more likely to be grammatically sound, its shorter length caps the potential number of accepted tokens per step, explaining the slightly lower overall performance metrics seen in Table 5.1.

Chapter 6

Conclusion and Future Directions

This chapter concludes the thesis by summarizing the research presented, acknowledging the limitations of the current Word-Group-Aware SAM framework, and outlining several promising avenues for future work. These future directions aim to address the current limitations by integrating linguistic knowledge more deeply into the core components of language models, from the tokenizer up to the training paradigm itself.

6.1 Summary of Research

This thesis introduced the Word-Group-Aware Suffix Automaton (WGA-SAM), a novel framework for speculative decoding tailored to the unique morphological and syntactic properties of Indian languages. We identified a critical gap in existing multi-token generation methods: their failure to respect the semantically cohesive "word groups" that are fundamental to Indian language grammar, as formalized by the Samanvaya framework.

Our proposed solution enhances the state-of-the-art SAM-Decoding methodology by introducing linguistic constraints directly into the drafting process. By constructing a Static SAM on a corpus annotated with Samanvaya word-group boundaries and modifying the draft generation algorithm to termi-

nate at these boundaries, our WGA-SAM ensures that all proposed token sequences are linguistically well-formed. This approach was designed to improve the quality of generated drafts, thereby increasing the acceptance rate and overall inference speed for Indian languages.

6.2 Limitations of the Current Work

While the WGA-SAM framework establishes a strong proof-of-concept for linguistically-informed drafting, its current implementation faces several key limitations that motivate the next phase of research:

1. **Dependence on a Static, Offline Corpus:** The linguistic intelligence of our system is entirely encapsulated within the Static SAM. Its performance is therefore fundamentally constrained by the size and diversity of the pre-annotated training data. As our results indicated, if a pattern is not present in this static corpus, our drafter cannot propose it, leading to a low match rate and diminishing the "voice" of our constrained drafter during inference.
2. **Inability to Perform Dynamic Word-Group Annotation:** The current framework's Dynamic SAM learns token-level repetitions but remains blind to word-group structure. The core challenge is the absence of a mechanism to detect word-group boundaries on-the-fly for newly generated text. This prevents us from creating a truly adaptive drafter that can learn and propose query-specific, linguistically-valid word groups.
3. **Rigidity of the Inference Loop:** The existing speculative decoding loop is unforgiving. A mismatch between the drafter's proposal and the verifier's prediction leads to rejection. Because our linguistic constraints are only applied within the Static SAM, the system has no mechanism to teach the main verifier model about the desirability of these structures, nor can it adapt its understanding of word groups during a single inference session.

6.3 Future Research Directions

The limitations of the current work open up several exciting and high-impact avenues for future research. We propose to move beyond simply constraining a retrieval-based drafter and instead integrate linguistic knowledge into the very fabric of the language model and its training process.

The most fundamental approach is to modify the tokenization process itself, teaching the model to "think" in terms of word groups from the very first layer. Drawing inspiration from recent work like SuperBPE [Liu et al., 2025] and BoundlessBPE [Schmidt et al., 2025], we can adapt the token-merging algorithm to favor the creation of "superword" tokens that correspond to Samanvaya word groups.

We propose exploring the following directions:

- **Direction 1: Constrained Merging:** Modify an algorithm like BoundlessBPE, which allows merges across whitespace only for complete words. Instead of this heuristic, we will permit a "superword" merge between two adjacent pre-tokens if and only if they belong to the same pre-annotated Samanvaya word group.
- **Direction 2: Group-as-Pretoken:** Instead of starting the BPE algorithm with whitespace-separated words as the base units, we will pre-process the corpus to treat each Samanvaya word group as an initial, inseparable pre-token.
- **Direction 3: Score Modification:** rather than using hard constraints, we can modify the standard BPE merge scoring function. The score for merging two tokens (a, b) would be a function of not only their co-occurrence frequency but also a bonus term that rewards the merge if (a, b) is part of a valid word group: $Score(a, b) = Freq(a, b) + *IsInGroup(a, b)$.

A second major research thrust involves redesigning the training paradigm to explicitly teach the model linguistic structure. Standard knowledge distillation (KD) suffers from a train-inference mismatch. We propose to leverage

an on-policy, self-distillation approach, inspired by recent works like SDFT [Shenfeld et al., 2026] and GKD [Agarwal et al., 2024].

The methodology would be as follows:

- **Teacher-Student Setup:** We use the same model for both the teacher and the student. The Teacher policy receives the input sentence plus the Samanvaya word-group structure as an additional conditioning variable c . The Student policy receives only the input sentence.
- **On-Policy Training Signal:** The student generates its own output sequence (a trajectory). The teacher then provides feedback on this student-generated trajectory.
- **Linguistically-Informed Loss Function:** The student is trained to not only match the teacher’s token probabilities (L_{KD}) but also to predict the linguistic structure that the teacher is aware of. The total loss would be a weighted sum of several components:

$$L_{\text{Total}} = L_{\text{KD}} + \lambda_1 L_{\text{boundary}} + \lambda_2 L_{\text{contrastive}} \quad (6.1)$$

- L_{boundary} : A boundary prediction loss, where the student predicts *True* or *False* for each token corresponding to a word-group boundary.
- $L_{\text{contrastive}}$: A contrastive loss that pulls the representations of tokens within the same word group closer together in the embedding space, while pushing representations of tokens from different groups apart.

This converts the offline, structural knowledge of Samanvaya into an on-policy learning signal, teaching the student model to generate text that is inherently structured into coherent word groups.

Bibliography

- [Agarwal et al., 2024] Agarwal, R., Vieillard, N., Zhou, Y., Stanczyk, P., Ramos, S., Geist, M., and Bachem, O. (2024). On-policy distillation of language models: Learning from self-generated mistakes.
- [Brown et al., 2020] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. (2020). Language models are few-shot learners. *CoRR*, abs/2005.14165.
- [Hu et al., 2024] Hu, Y., Wang, K., Zhang, X., Zhang, F., Li, C., Chen, H., and Zhang, J. (2024). Sam decoding: Speculative decoding via suffix automaton.
- [Kamath et al., 2025] Kamath, A., Singla, K., Paul, R., Joshi, R. B., Vaidya, U., Chauhan, S. S., and Wartikar, N. (2025). Benchmarking Hindi LLMs: A new suite of datasets and a comparative analysis. In Bhattacharya, A., Goyal, P., Ghosh, S., and Ghosh, K., editors, *Proceedings of the 1st Workshop on Benchmarks, Harmonization, Annotation, and Standardization for Human-Centric AI in Indian Languages (BHASHA 2025)*, pages 52–68, Mumbai, India. Association for Computational Linguistics.
- [Karthika et al., 2025] Karthika, N., Patra, A., Naidu, N. S., Bhattacharya, A., Ramakrishnan, G., and Dangarikar, C. (2025). Semantically cohesive word grouping in indian languages. *arXiv preprint arXiv:2501.03988*.

- [Khan et al., 2024] Khan, M. S. U. R., Mehta, P., Sankar, A., Kumaravelan, U., Doddapaneni, S., G, S., G, V. B., Jain, S., Kunchukuttan, A., Kumar, P., Dabre, R., and Khapra, M. M. (2024). Indicllmsuite: A blueprint for creating pre-training and fine-tuning datasets for indian languages. *arXiv preprint arXiv: 2403.06350*.
- [Leviathan et al., 2023] Leviathan, Y., Kalman, M., and Matias, Y. (2023). Fast inference from transformers via speculative decoding.
- [Li et al., 2025] Li, Y., Wei, F., Zhang, C., and Zhang, H. (2025). Eagle: Speculative sampling requires rethinking feature uncertainty.
- [Liu et al., 2025] Liu, A., Hayase, J., Hofmann, V., Oh, S., Smith, N. A., and Choi, Y. (2025). Superbpe: Space travel for language models.
- [Schmidt et al., 2025] Schmidt, C. W., Reddy, V., Tanner, C., and Pinter, Y. (2025). Boundless byte pair encoding: Breaking the pre-tokenization barrier.
- [Shenfeld et al., 2026] Shenfeld, I., Damani, M., Hübotter, J., and Agrawal, P. (2026). Self-distillation enables continual learning.
- [Touvron et al., 2023] Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., and Lample, G. (2023). Llama: Open and efficient foundation language models.
- [Xia et al., 2024] Xia, H., Yang, Z., Dong, Q., Wang, P., Li, Y., Ge, T., Liu, T., Li, W., and Sui, Z. (2024). Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In Ku, L.-W., Martins, A., and Srikumar, V., editors, *Findings of the Association for Computational Linguistics ACL 2024*, pages 7655–7671, Bangkok, Thailand and virtual meeting. Association for Computational Linguistics.